Question	Answer
sir how to define layers in our model	A layer is a building block of a neural network that transforms input features into new representations by applying learned parameters (weights and biases) and often an activation function. Like in Simple neural network we have one or two layer of NN
Everytime we tune hyperparam , we need to re train the model?	yes.
how do we evalutate in validation set ?	During validation, we use the trained model to make predictions on unseen validation data and measure performance using metrics such as loss, accuracy, or MAE. The model weights are not updated during this step.
During Validation Process, when model is again hypertuned and reupdated, how this entire process occurs when training already started ? looking to understand this technically. Also, when model is again tuned, does it need to train from beginning?	During validation, the model is not retrained or updated. It only makes predictions on the validation data, and we measure performance (loss, accuracy, etc.). The weights remain unchanged.
Is there is a loop of training to validation and validation to training for getting the good model?	Yes. Training involves a loop: train for an epoch, evaluate on validation data without updating weights, and stop when validation performance plateaus (early stopping).
Aren't we loosing out on learning data by dividing test and validation?	Yes we do lose some data for training by creating validation and test sets. However this trade-off is necessary. Without separate validation and test sets we cannot reliably know if the model is actually learning or just memorizing the training data. How we minimize the loss: With large datasets the percentage impact is small so we can still afford 10-20% for validation + test. Use k-fold cross validation — this way every data point gets used for both training and validation across different folds. In very large production datasets we often use 95-5 or 90-10 splits.
What is the difference between training error and validation error?	Training error is the error calculated on the data the model was trained on. It shows how well the model fits the seen data. Validation error is the error calculated on the separate validation set that the model has not seen during training. It shows how well the model generalizes to new data. Key difference: Low training error + low validation error → good model Low training error + high validation error → overfitting (model memorizing training data)
do these concepts for LLMs as well?	For LLMs, validation is often performed periodically (e.g., every few thousand training steps). The model is switched to evaluation mode, run on the validation set, and metrics such as validation loss or perplexity are computed. Then training resumes from the current weights.
If overfitting has too much noise, how can more data fix it ?	Overfitting happens when the model learns both the real pattern and the random noise present in the limited training data. With small data, noise becomes a big part of what the model sees. When you add more data: The true underlying signal becomes stronger and clearer. Random noise gets averaged out across many examples. The model is forced to learn general patterns instead of memorizing specific noise. This improves generalization. More high-quality data is one of the most effective ways to reduce overfitting.

what are hyper paramters	Hyperparameters are settings that you choose before training starts. They control how the model learns, but are not learned from the data. Like - Learning rate,Batch Size, Number of Epochs etc
Follow up: During validation, the model is not retrained or updated. It only makes predictions on the validation data, and we measure performance (loss, accuracy, etc.). The weights remain unchanged. Question: If model is not retrained and we know its not giving predictions- whats the point of validation as there is cost involved during training?	Validation is used to detect overfitting, compare different hyperparameter settings, and determine the optimal point to stop training.
Is this percentage of 80-10-10 fix or depends on data type/model	It depends 80-10-10 is just a common default, and the actual split should be guided by dataset size, task type, and benchmark conventions.
how to identified underfit ,overfit or goodfit model (neeed parameter value)	using the validation and train set, if performance good in both sort of good fit. If bad in both underfit and if good in train set and bad in validation set then overfit
why underfitting is called high bias?	The purpose of validation is to check whether the model can perform well on new, unseen data and not just on the data it was trained on.
what is exact difference between bias and variance, underfitting gives more error, overfitting also gives more error. so both have high bias and variance?	High bias leads to underfitting (high error on all sets). High variance leads to overfitting (low training error but high validation error).
When we provide the data for training, do we seggregate it as noise and actual data before feeding to the model ?	No. We usually do not manually segregate noise and actual data before training. In real projects we feed the data as it is after basic cleaning. The model is expected to learn the underlying signal and ignore random noise.
What are examples of real-world problems where underfitting is more common than overfitting?	Underfitting is common when using overly simple models for complex tasks, such as using linear regression for highly non-linear data.
wanted to understand how and why the model decides the classifications using the given data. For example, if we have a problem where 10 different classifications need to be predicted, how does the current model handle it? What is the decision-making process behind choosing a particular classification? Could you please explain this with an example?	In multi-class classification with 10 categories, the model learns patterns from labeled training data. How it decides: During training, the model adjusts its weights to map input features to one of the 10 classes. At the end, it uses Softmax activation in the output layer. This converts raw scores into probabilities for all 10 classes that add up to 1. The class with the highest probability is chosen as the prediction.
wanted to understand how and why the model decides the classifications using the given data. For example, if we have a problem where 10 different classifications need to be predicted, how does the current model handle it? What is the decision-making process behind choosing a particular classification? Could you please explain this with an example?	The model learns patterns from the training data and assigns a score (or probability) to each class. The class with the highest score is chosen as the prediction. Example (10 classes): Suppose we are classifying images into 10 categories (cat, dog, car, plane, etc.). For a new image, the model might output: Cat: 0.70 Dog: 0.15 Car: 0.05 Plane: 0.03 Other classes: remaining probability Since Cat has the highest probability (0.70), the model predicts Cat. During training, the model adjusts its weights so that images of cats get higher scores for the "Cat" class, dogs for the "Dog" class, and so on. Over time, it learns which features are useful for distinguishing between the classes.

The purpose of validation is to check whether the model can perform well on new, unseen data and not just on the data it was trained on. Then overfitting should be high bias. why underfitting?	No. Overfitting means high variance, not high bias. Underfitting is when model is too simple which signifies high error on both training and validation data -> high bias. Overfitting -> model memorizes training data -> low training error but high validation error -> high variance. Validation helps detect overfitting by testing performance on unseen data.
Is number of weights a hyperparameter or it core model design?	Number of weights is part of core model design, not a direct hyperparameter. It is decided by your architecture choices: Number of layers Number of neurons in each layer Type of layers (dense, convolutional etc.)
can you explain more on what is high variance	High variance means the model learns the training data too closely, including noise, instead of learning the general pattern. As a result: Training performance is very good. Validation/test performance is poor. The model does not generalize well to new data.
If model is expected to learn then how does it know in the overfitting situation that the noise isnt the actual data ?	The model cannot tell whether a pattern is real or just noise. It learns whatever reduces training error. We use the validation set to check if those learned patterns also work on unseen data. If not, the model is likely overfitting.
what does regularization mean, is it to handle underfitting or overfitting?	Regularization is used to reduce overfitting. It prevents the model from becoming too complex and memorizing the training data. Too much regularization can cause underfitting.
what is weight? I missed what professor earlier said.	weight is a number that determines how important an input feature is to the model's prediction. During training, the model learns the best values of these weights to make accurate predictions. Like simple NN y = wx + b,here w is the weight which will be learned during training.
any formula for bias and variance?	Bias is the difference between our model's average prediction and the correct, true value. High bias means our model is making overly simplistic assumptions, which leads to underfitting. So you can formulate Bias of our f_pred(x) = E[F_pred(x)] - f(x) similarly Variance measures how much our model's predictions would jump around if we gave it a different set of training data. High variance means the model is memorizing the noise, which leads to overfitting. formulate it something like Var(f_pred(x)) = E[(f_pred(x) - E[f_pred(x)])^2] Hope this is clear :)
Batch Normalization not clear	Batch Normalization keeps the values flowing through the network within a consistent range. This prevents some layers from receiving values that are too large or too small, making training faster and more stable.
Dropout is done by the model automatically or do we do the same manually?	We add Dropout manually while designing the neural network architecture. During training, the framework automatically switches off a random subset of neurons in each iteration. During inference, all neurons are used. Dropout is a regularization technique that prevents the network from becoming too dependent on specific neurons, helping reduce overfitting.
Dropout is described as a regularisation technique but at inference time it is switched off. How does that prevent overfitting during training?	During training, dropout randomly switches off some neurons, forcing the network to learn more robust features instead of relying too much on specific neurons. This reduces overfitting. During inference, dropout is turned off and all neurons are used (with appropriate scaling already accounted for during training).

Is my model learning correctly or just memorizing??	This is exactly what the validation set helps us determine. If the model performs well on both training and validation data, it is likely learning useful patterns. If it performs well only on training data but poorly on validation data, it is likely memorizing (overfitting).
Also, in neural networks, if the bias is a higher number then the w(i)x + b would still give a higher value even if weight is low, isnt it ? How do we solve this problem ?	Yes, a large bias can increase the neuron's output even when the weights are small. However, the bias is also a learnable parameter, just like the weights.
If L1 gives importance to some weights than others is it also random or a design choice? and if random, is Dropout an extreme case of L1?	L1 regularization adds a penalty to large numbers of weights and naturally pushes less useful weights toward zero during training. The model decides which weights are important based on the data. Dropout is random. During training, it temporarily switches off random neurons to prevent over-reliance on specific paths.
Sir said dropout randomly switches off weights during training. But if we're switching off neurons, aren't we training a different network each time? How does the final model actually work then?	Yes. During training, dropout effectively trains many different subnetworks by randomly switching off neurons. However, all these subnetworks share the same weights. During inference (testing), dropout is turned off and the full network is used, with activations appropriately scaled. This acts like averaging the predictions of many subnetworks, which improves generalization and reduces overfitting.
What's meaning of diminishing returns ?	Diminishing returns means that after a certain point, putting in more effort, data, or resources leads to smaller and smaller improvements.
What is regularization and how is it different from under and over fitting?	Regularization is used to reduce overfitting. It prevents the model from becoming too complex and memorizing the training data. Too much regularization can cause underfitting.
Can a model simultaneously show signs of both overfitting and underfitting in different regions of the feature space?	Yes. A model can overfit some parts of the data and underfit others. For example, if there is plenty of data for one customer segment but very little for another, the model may learn the first segment extremely well while failing to capture patterns in the second.
Is good fit a sinusoidal graph always?	No, it will depend on the data distribution. how the data is distributed and what the model learns...
Is there any parameter that model returns to determine if model is under/good/over fitted? based on the validation data?	No. There is no single parameter for this. We usually compare training and validation error/loss: Underfitting: High training loss and high validation loss. Good fit: Low and similar training and validation loss. Overfitting: Low training loss but high validation loss.
how are we plotting these graph , what is the x axis and y asis ?	the example is a simple sin cureve with x axis as input theta and y axis as the output y = sim theta
so we plot bias variance to see the error and then at the lowest dip we train our model?	Not exactly. You don't usually plot a bias-variance curve and then train at the lowest point. Instead, you train multiple versions of the model with different complexities and evaluate them on validation data. The bias-variance tradeoff is a conceptual way to understand what's happening
Also, Is minimizing bias always a sweet spot?	No. A very low-bias model can become overly complex and overfit the training data, leading to high variance. The objective is to find a good bias–variance tradeoff that generalizes well to unseen data.

will validation data set also comes under unseen data set ?	Yes — validation data is "unseen" by the model's weights, but indirectly seen by you (the modeler) through repeated hyperparameter tuning, making it slightly "contaminated." Only the test set is truly unseen. Your analogy nails it: Train → Textbook + Homework Validation → Practice Quiz (you keep adjusting strategy based on it) Test → Final Exam (completely blind)
if both Underfitting and Overfitting give poor results, how can I identify which one my model has?	Look at both training and validation performance. Underfitting: Poor performance on both training and validation data. Overfitting: Very good performance on training data but poor performance on validation or test data.
can you give one real-world example where overfitting caused problems?	Suppose a model is trained to detect cancer from X-ray images. If most cancer images in the training set come from one hospital and contain a specific scanner artifact or hospital watermark, the model may learn to associate that artifact with cancer instead of learning actual disease patterns. As a result, it achieves very high training accuracy but performs poorly on X-rays from other hospitals. This is a classic case of overfitting, where the model memorizes dataset-specific patterns rather than learning the true medical features.
why is Underfitting called High Bias and Overfitting called High Variance?	High Bias = Underfitting because the model makes strong simplifying assumptions and fails to capture the true patterns in the data. High Variance = Overfitting because the model becomes too sensitive to the specific training data and its predictions can vary significantly with small changes in the data
f there are large weights, this would increase to the loss?	Not necessarily. Large weights do not automatically increase the loss in a neural network. The loss is calculated based on how far the predictions are from the true values. If large weights produce accurate predictions, the loss can still be low.
f my Training Accuracy is 98% and Validation Accuracy is 65%, what should I conclude and what should be my next step?	overfitting. Use regularization (L1/L2). Add/dropout or increase dropout. Reduce model complexity (fewer layers/parameters).
Can a model simultaneously overfit on some classes and underfit on others in a multi class classification task?	Yes. In a multi-class classification problem, a model can overfit some classes while underfitting others.
Is embeded value and weights are same?	Not exactly. Embeddings are learned vector representations of inputs (e.g., words, tokens, images). Weights are the parameters used throughout the neural network to transform and combine inputs. An embedding matrix itself contains trainable parameters (weights), but embeddings are a specific type of learned representation, not all weights.
again how we will define hidden layer	A hidden layer is any layer between the input layer and the output layer. Input Layer -> Hidden Layer -> Output Layer The hidden layer receives inputs, applies weights, bias, and an activation function, then passes the result to the next layer. Its called hidden because we do not directly observe its values in the input data or the final prediction. It learns intermediate representations internally.
Where do we actually use Lasso Ridge and combination of it	Lasso selects important features, Ridge stabilizes the model by shrinking weights.

Also, Is minimizing bias always a sweet spot?	No. Minimizing bias alone is not the goal. As you reduce bias by making the model more complex, variance often increases. Reducing bias is good up to a point, but if you keep increasing model complexity, you may start overfitting. The sweet spot is a balance between bias and variance, not the minimum possible bias.
AttributeError Traceback (most recent call last) /tmp/ipykernel_1667/2669310536.py in <cell line: 0>() 1 from sklearn.linear_model import Ridge, Lasso 2 ridge=Ridge(alpha=1.0) ----> 3 print(ridge.coef_) 4 5 lasso=Lasso(alpha=0.5) AttributeError: 'Ridge' object has no attribute 'coef_'	you need to train the model first. this was just a code snippet sample on how to use the ridge try: from sklearn.linear_model import Ridge, Lasso ridge = Ridge(alpha=1.0) # Train the model first ridge.fit(X_train, y_train) # Now coefficients are available print(ridge.coef_)
if neurons didn’t drop then what? pros and cons? it depends on case or somethi ng else?	If neurons are not dropped (i.e., no dropout is used), the network can use all its capacity to learn patterns from the data. Pros: Can learn faster. May achieve higher training accuracy. Cons: Higher risk of overfitting. The network may rely too much on specific neurons/features.
how to figure out model is over fitting or underfitting	Underfitting: Poor performance on both training and validation data. Overfitting: Excellent performance on training data but significantly worse performance on validation data.
what is dropout in layman language	Dropout is like randomly asking some team members to sit out during practice. In each training step, some neurons are temporarily switched off, forcing the remaining neurons to learn useful patterns on their own instead of relying too much on specific neurons. This helps the model become more robust which in turn reduces overfitting.
Can you provide an example where dropout may be important?	Dropout is crucial when training large Deep Learning models (like image or text classifiers) that are prone to overfitting. Very small example Imagine training a neural network to identify images of "Dogs." Without dropout, the network might realize that many dog pictures in your training set have grass in the background. A few specific neurons might memorize the "green grass" pixels and use that to predict "Dog," entirely ignoring the actual shape of the animal. By randomly turning off (dropping out) a percentage of neurons during training, the network is literally unable to rely solely on its "green grass detector" neurons. It is forced to distribute its learning and pay attention to other features, like the shape of the ears, the snout, or the fur. This creates a much more robust model that generalizes well to unseen data.
Machine tiggers others neuron to learn about the shape. Is it when that particular neuron fails or its default behavior for all neurons to learn about shape from when training starts?	With dropout, neurons are randomly switched off in different training iterations. Because any neuron might be unavailable, other neurons are forced to learn useful features as well instead of relying on a single neuron. Final takeway: From the start of training, the network learns to distribute information across multiple neurons, making it more robust and less likely to overfit.

Is embeded value and weights are same?	Not exactly. Embeddings are learned vector representations of inputs (e.g., words, tokens, images). Weights are the parameters used throughout the neural network to transform and combine inputs. An embedding matrix itself contains trainable parameters (weights), but embeddings are a specific type of learned representation, not all weights.
If redundant repetition is happening, then whole of the idea of one neuron performing on task and being agnostic of what other neurons are doing is lost isnt it ?	YES! This is exactly why techniques like Dropout were introduced.
Wont creating redundancy create duplication and decision making capability impaired, as they can give slightly different information	Some redundancy is beneficial because it makes the network less fragile. The problem arises only when there is excessive redundancy, where many neurons learn nearly identical features and add little value.
f neurons are not dropped (i.e., no dropout is used), the network can use all its capacity to learn patterns from the data. Pros: Can learn faster. May achieve higher training accuracy. Cons: Higher risk of overfitting. The network may rely too much on specific neurons/features. can you please explain with real world simple if possible ? i didn't get exactly	Without Dropout Suppose only one player is extremely good at scoring goals. The whole team starts relying on that player. The team wins many practice matches (training data). But if that player is injured on match day (new unseen data), the team performs poorly. This is like overfitting: the network depends too much on a few neurons. With Dropout During practice, the coach randomly asks some players to sit out. Sometimes the star striker sits out. Other players are forced to learn how to score and create chances. The team becomes more balanced and robust. Now, even if one player is unavailable, the team can still perform well.
how and who decides how many hidden layers will be there?	The ML engineer decides, but it's a hyperparameter chosen through experimentation — start with 1-2 layers, add more if underfitting, stop when overfitting begins. For complex tasks, just reuse proven architectures (ResNet, Transformers) where researchers already figured it out.
Why have we sorted the X data set?	X_data is not sorted it is a random split with a 80-20 set
Purpose of StandardScaler vs Ridge?	StandardScaler scales features to a similar range (usually mean = 0, standard deviation = 1).
what is the input of this program?	training set -> feaures and target values.
Why cross-validation is able to update the weights, it should not ideally..??	Cross-validation itself does not update weights on the validation fold. What happens is: Split data into folds. For Fold 1: Train the model on the training folds → weights are learned. Evaluate on the validation fold → weights are NOT updated. For Fold 2: A new model is trained on a different set of training folds. Evaluate on a different validation fold.
is there a thumb rule or general pattern regarding number of weights and total number of neurons and layers. DO more of later give better model?	There is no fixed thumb rule, but there is a general pattern. More layers and neurons increase the model's ability to learn complex relationships. However, beyond a point, adding more can lead to overfitting, longer training times, and higher computational cost.

When regularization part will come, before cross validation or after cross validation ?	Regularization is part of the model training process. Cross-validation is used to find the optimal amount of regularization. So regularization comes during training, while cross-validation helps tune its strength.
If I use cross validation to tune hyperparameters, can I still overfit even though I never directly touched the test set?	Yes. This is called "Overfitting to the Validation Set" or "Information Leakage." Even though the algorithm isn't directly training on the validation data, you are using the validation scores to choose your hyperparameters. If you run a massive Grid Search and test thousands of different hyperparameter combinations, you are basically rolling the dice thousands of times. Eventually, just by pure statistical luck, you will find a combination that perfectly aligns with the random noise in your validation folds. Small analogy would be - If you take the exact same practice test 1,000 times, and change your study strategy every time just to get a higher score on that specific test, you aren't actually learning the material anymore. You are just memorizing the practice test. When you finally sit down to take the final exam (the true Test Set), you will still fail.
In case of Dropout ,one neuron is switched of..say for example a neuron which is havig shape information..Now after Dropout other neurons will provide the Shape infomation ?	If a "shape-detecting" neuron is dropped during training, the network cannot rely on it for that training step. As a result, other neurons are encouraged to learn shape-related information as well. Final key takeaway: Instead of one neuron being solely responsible for detecting shapes, multiple neurons learn to contribute to shape detection.
On this slide, why are we doing cross validation after checking score on test data? Shouldnt it be done as part of training?	In practice, cross-validation is usually performed before the final test evaluation. In the slides, the test score is shown first to demonstrate model performance.
CV by doing it ,are exposing data to test?.explain more on this	It only splits the training data into different train and validation folds. A sample may be used for training in one fold and validation in another, but the test set remains completely untouched until the final evaluation.
The weights can be of different magnitudes if not scaled. Scaling is a must for L2 regularization? What if my scenario don't want to scale features?	L2 regularization works best when features are scaled. If you choose not to scale, L2 may penalize some features more than others simply because of their units of measurement, not because they are less important.
The weights can be of different magnitudes if not scaled. Scaling is a must for L2 regularization? What if my scenario don't want to scale features?	In 99% of practical applications, Yes we need to do scaling. L2 Regularization works by hunting down large weights and shrinking them. If you cannot scale your features for business or interpretability reasons, you have other options: 1. Switch to tree models: Random Forest, XG Boost, LightGBM etc. 2. Custom Feeature penalties: Standard L2 applies one global penalty to everything. In advanced math you can manually assign a unique penalty strength to each individual feature based on its variance
Do we use L2 or L1 right from start of the training or after observing the loss in actual and predicted output? Are these to be always used as prevention mechanism?	L1/L2 are typically chosen before training as regularization techniques. They are used when overfitting is a concern, not necessarily in every model.

What is early stopping? What's its use case in real life?	Early stopping means stopping training when the validation performance stops improving, even if more epochs are remaining. Why use it? It helps prevent overfitting. Example: Epoch 10: Validation accuracy = 90% Epoch 15: Validation accuracy = 92% Epoch 20: Validation accuracy = 91% The best model was at Epoch 15, so we stop there instead of continuing to train and overfit.
what is high variance?	High variance means the model learns the training data too closely, including noise, and therefore does not generalize well to new data.
so even validation data also touched here for training..then why do we split training and validation	Validation data provides an unbiased check of model performance during each training run.
whats the mlp ?	Multi layer perceptron, simple feed forward neural network. y= wx +b , w is weight and b is bais
L2 Regularization reduces dependency on existing data..can you please explain this a bit	L2 regularization shrinks large weights, preventing the model from relying too much on a few features and helping it generalize better.
but during CV=5 even thistraining and validation set is splited in to 5 parts and train and test on each part. is my understanding right?	In 5-fold cross-validation, the training data is split into 5 parts. The model is trained 5 times, each time using 4 folds for training and 1 fold for validation. The final performance is the average across all 5 runs.
what does “spatial pattern” mean wrt CNN	A spatial pattern is a meaningful arrangement of pixels or features in an image, such as edges, shapes, or objects, that a CNN learns to recognize. For example in a picture of dog the special features will be Eyes, nose, fur color etcs.
Is ReLU actually a sigmoid fun ?	Sigmoid Produces values between 0 and 1 Has an S-shaped curve Can suffer from vanishing gradients ReLU Outputs 0 for negative inputs Outputs the input itself for positive inputs Simpler and computationally faster
Is ReLU actually a sigmoid fun ?	No, relu is z = max(0,x)
what is class in stacking layers?	The stacked layers (convolution, pooling, fully connected) learn features, and the final output layer produces probabilities for each class.
Does CNN require GPU + Data instead of a high power CPU?	Yes
In CNN, convolution + RELU done manually or by model during training phase? As its written as feature extraction	by the model, prof. will explain the stages in details. in DL, the feature extraction is not done by us instead its learned by the model
Like if I have a high computing CPU like M5 Max Chip, will I not be able to develop a CNN?	Yes you can use M5 max chip, but it will take around 3 to 4 days of training. But high end GPUs like A100 and H100 will run the few hours. So it depends on model size and data size, if data size is small and model is simple CNN then M5 is usefull, but you have larger data and more deep CNN then GPU is better than M5.
How to decide how many layers of Convolution +ReLU is required before flattening?	There is no fixed rule. The number of Conv + ReLU layers depends on the complexity of the task and the size of the dataset. A common approach is to start with a simple architecture and add more layers only if validation performance improves. More layers can learn more complex features, but they also increase the risk of overfitting and training cost.

how does CNN know that a group of edges becomes an eye & a group of eyes, ears, and nose becomes a cat?	the weights on the conv layers are learned in the process.
So for whole image all its features will be divided into kernels and then the layer and all this process is done for all features	A kernel slides across the entire image and extracts features such as edges and shapes. Multiple kernels learn different features, and this process is repeated across layers to learn more complex patterns.
when CNN looks at an image does it learn the filters automatically or do we manually define them???	We define the structure of the filters, but the filter values themselves are learned automatically from the training data. The CNN discovers which patterns are useful for the task.
so class is collection of stacked layers?	A model class is usually a collection of layers connected together to perform a specific task, such as classification or prediction.
why can't we use a normal Neural Network (MLP) for images? Why do we need CNN??	MLPs only accept data in a single, straight 1D line. To feed a 2D image into an MLP, you have to cut it up row by row and string it together into a single flat line. In an MLP, every single pixel must connect to every single neuron in the next layer. Even for a tiny, low-resolution color image (like 100x100 pixels), that is 30,000 input values. If your first hidden layer has just 1,000 neurons, the network has to learn 30 Million weights just in the first step! The model will require massive computing power, run incredibly slow, and almost certainly overfit. CNNs use "Parameter Sharing" (the same 3x3 filter sliding over the whole image), drastically reduces the math required.
what exactly is a kernel/filter? Can you show one simple example???	A kernel is a small matrix that slides across an image to detect patterns like edges, textures, and shapes. Different kernels learn to detect different features.
What is ReLU, can you explain more?	ReLU is the activation function (Lecture 1)
Is the Convolution matrix the same as adding weights to a neuron?	Yes, conceptually. A convolution kernel is a small set of learnable weights, similar to the weights of a neuron. The difference is that the same kernel weights are reused across the entire image as the kernel slides over it.
How filter values are determined?	they are parameters, learned in the process
How this kernel/filter derived?	Kernel/Filter - It is learned automatically during training. As the CNN processes images: It makes predictions. The loss is calculated. Backpropagation determines how each filter contributed to the error. The filter values are updated to reduce the error.
How this kernel/filter derived?	They are not manually derived by humans They are learned automatically by the network through a process called Gradient Descent and Backpropagation. When you first build a CNN, the network fills every single 3x3 filter with completely random numbers. At this point, the filters are useless and look for nothing but static noise. You feed a picture of a cat into the network. The random filters scan the image and spit out a completely wrong prediction The network compares its terrible guess to the true answer and calculates the mathematical difference. This difference is called the Error or Loss. Now the Backprop - The network uses calculus to trace that massive error backward through the layers. It looks at every single number in every single filter and asks: "Did this specific number make the error better, or worse?" so like this the filters or kernel numbers are learnt

how kernel values are defined ? on what basis ?	similar to how we defined the weights of the MLP, he values are parameters
and if a bias is attached to it, positive value indicates the activation ?	Yes. The bias shifts the result of the convolution (or neuron output). A positive bias tends to increase the activation value, making it more likely that the neuron will activate (especially with activations like ReLU). However, activation depends on the combined effect of weights, inputs, and bias, not the bias alone.
how to get the kernel?	the values are learned -> parameter set
isnt dot product row X column for each element, It see row X row, what am i missing?	Yes, convolution involves a dot product, but not just a single one. As the kernel slides across the image, a dot product is computed at each position, producing a feature map.
In the example input image pixel is 0, 1 only how does matrix has other positive number	The input image may contain only 0s and 1s (or pixel values 0–255), but the feature map values are produced after convolution.
so the filter is taking 3X3 part from input image and making a feature map with dot product. but how we decided filter values?	Filter values are not chosen manually. They start with random values and are automatically updated during training to learn useful patterns from the data. These are like y = wx +b, where X is image and W is the learnable kernal.
What is both 1st column and 3rd column have high values? That means there are sharp edges both sides, but feature map will be close to 0?	Yes. Even if both sides have high values, the feature map can be near zero because the filter detects a specific pattern (contrast), not just large pixel values.
why big numbers mean straight line?	from the image perspective you can imagine that the big numbers are sort of having the same color so will form like the line as in the example case. as the adj ones were having less values
Hwo we decide filter matrix values ?	We do not decide the filter matrix values manually. The values are initialized randomly by the framework and then learned during training.
Do we have any default value for Filter/Kernal to start with? and how to decide whether 3x3 or 2X2 matrix to be used for calcualtion?	Filter values start with random initialization and are learned during training. The filter size is a design choice, with 3×3 being the most widely used because it provides a good balance between capturing patterns and computational efficiency.
The feature map step also involves neural network apart from MLP?	No. The creation of a feature map is done by the convolution operation (kernel + bias + activation), not by a separate neural network or MLP. MLP head is for classification
What is pooling layer? Is it memory pooling?	NO. A pooling layer reduces the size of the feature map while keeping the most important information. A pooling layer downsamples the feature maps by summarizing nearby values, often using the maximum value. It reduces the amount of data the network needs to process while retaining the most important features.
How to we get input map from image - is it just storing some image to vector or some model/function we have to apply first	No extra model is needed first. The image pixels themselves form the initial feature map, and CNN layers automatically learn higher-level feature maps from it.
Is MaxPool is differentiable? How will gradient descent work if we have maxPooling layer?	Gradient descent works because the network remembers which element was the maximum during the forward pass and routes the gradient back only through that element.
when we decide a slideing window we do not repeat the values that we have alreday considered?	In CNNs, sliding windows usually overlap, so the same pixel values are often used multiple times by different kernel positions.
pls provide more insight on feature map before pooling. What exactly large number suggest vs small numbers?	Think of a feature map value as a "match score" for what the kernel is looking for. A large value means: "Yes, I can clearly see this pattern here." A small value (close to 0) means: "I don't really see that pattern here." A negative value means: "This looks opposite to what I'm looking for." (before ReLU removes it). For example, if the kernel is detecting vertical edges, a high number at a location means there is a strong vertical edge there, while a low number means there isn't much of a vertical edge in that region.

what is stride 2 here?	Stride 2 means the kernel moves 2 pixels at a time instead of 1 while sliding across the image. Stride = 1 → kernel moves 1 pixel each step. Stride = 2 → kernel skips every other pixel and moves 2 pixels each step. This reduces the size of the output feature map and decreases computation.
please can you explain pooling layer , on why max value is picked??	MaxPooling keeps the strongest feature response in a small region, assuming that the most important information is where the feature is detected most strongly. You can take mean pooling also.
pls provide more insight on feature map before pooling. What exactly large number suggest vs small numbers?	Before pooling, a feature map shows how strongly a filter detects its target pattern at each location. Large numbers mean a strong match; small numbers mean a weak or no match.
what is softmax probability n how it works?	Softmax converts the model's output scores into probabilities that add up to 100%.
How the weights, feature map, kernel values are calculated?	weights are learned, kernel values are the weights that will be learned, feature maps are when we pass the image through a conv layer.
Are we running MLP post-flattening?	yes to classify the image in this case
Apart from ReLU what other activations do we have ?	many sigmoid, tanh, etc (refer Lecture 1)
What is stride 2 means?	Stride 2 means the kernel moves 2 pixels at a time instead of 1 while sliding across the image. Stride = 1 -> kernel moves 1 pixel each step. Stride = 2 -> kernel skips every other pixel and moves 2 pixels each step.This reduces the size of the output feature map and decreases computation.
so model used to learn feature map is it different than one used for classification?	Yes. Convolution layers learn feature maps, while later classification layers use those features to make the final prediction.
Can we say that because of maxpooling the CNN is translation invariance..???	Yes, partially. MaxPooling helps CNNs become more translation-invariant, meaning small shifts in an object's position do not change the output much.
How does Max pooling vs avg pooling impact training	Max pooling often works better for feature detection, while average pooling provides a more generalized representation.
is CNN still used and needed for any Images like scenarios for example pattern detection, face detection, similar image , comparisions of image - In all such scneraios the Model will use CNN ?	Yess more complex versions of CNN like YOLO, ImageNet, AlexNet, now days VLLMS.
How the weights, feature map, kernel values are calculated from the image?	weights are learned, kernel values are the weights that will be learned, feature maps are when we pass the image through a conv layer.
is the MLP at the end already pretrained on something or does it train on the flattened data	it is trained on the classification task
Is Scikit learn outdated and we're not using it anymore?	Yess traditional ML uses Scikit Learn, but now a days PyTorch as used widely by big tech gaints like MeTA, Google, MIT.
Do industries prefer PyTorch that scikit libraries?	It depends on the problem. For traditional machine learning problems, Scikit-learn is still heavily used in industry. For deep learning, computer vision, NLP, and GenAI, PyTorch is generally preferred today.
What are the main components of PyTorch???	PyTorch mainly consists of Tensors, Autograd, Neural Network modules (nn), Optimizers, Loss Functions, and Data Loading utilities.
why padding is used ?	to increase the dimensions of the image for the next conv layer
How are we defining the number of output feature maps in CNN?	The number of output feature maps is chosen by the designer as the out_channels parameter of the convolution layer.

How do we determine the feature map count here?	nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3)
Is it on intuition that we select activation functions ?	Not purely intuition. It is based on experience, theory, and what has worked well in practice. Some common choices are: ReLU for hidden layers in most deep networks Sigmoid for binary classification output Softmax for multi-class classification output Tanh in some specialized architectures
so here eigen value and vector wil be used? during simple CNN forward pass?	No. A standard CNN forward pass does not use eigenvalues or eigenvectors; it mainly uses convolutions, activations, pooling, and linear layers. These are used in PCA.
How did the input come to 1x28x28?	Its our image Input for this CNN usecase, 1 grayscale channel with 28x28 image, these are provided images for our classification problem.
Size of the matrix should reduce after the convolution layer right (n -> n -2, m -> m -2 in case kernel size is 2)	For a 2×2 kernel, the size becomes ((n-1) * (m-1)), not ((n-2) * (m-2)). The exact output size depends on the kernel size, stride, and padding.
How do we decide how many round of convolutions are required?	The number of convolution layers is chosen by the model designer. More layers can learn more complex features, but also increase computation and the risk of overfitting.
the secind level of convulation the output of first pass goes as input..? If yes, we were loooking for one info in the first pass so we might have removed other info in the pooling how will the other features analyzed from this input in the second conv layer	Yes. The output feature maps from the first convolution layer become the input to the second convolution layer. Some information is lost during pooling, but pooling is designed to keep the most important features. The second convolution layer does not look at the original image; it learns higher-level patterns by combining the features preserved from the first layer (e.g., edges - > shapes -> objects).
and what is block 1 and 2 representing? in slide	Block 1 and Block 2 are usually groups of layers (e.g., Conv + ReLU + Pooling). Block 1 learns simple features like edges and textures. Block 2 learns more complex features like shapes and object parts.
how to define logits in CNN ot it will be calculated at end?	Logits are the raw outputs of the final layer before an activation function such as Softmax or Sigmoid is applied. You do not manually define the logits. You define the final layer, and the network calculates the logits automatically.
So round of convolutions needs to hard coded with it's features for every round?	We hard-code the architecture (layers and filter counts), but the features learned in each convolution layer are discovered automatically from the data. nn.Conv2d(3, 32, 3) nn.Conv2d(32, 64, 3) nn.Conv2d(64, 128, 3)
Can we say the different kernel matrix define different features?	Yes. Different kernels learn different features, which is why CNNs use multiple filters in each convolution layer.
How can a single scalar value or a set of values produced by a convolutional layer indicate which object is present in an image? My understanding is that the output of CNN layers consists of feature maps, which are eventually transformed into vectors or scalar values. If a value is high, we might interpret it as the detection of a particular feature, such as an edge or texture. However, how does the model move from detecting low-level features like edges, corners, and patterns to recognizing a complete object such as a cat or a dog? In other words, how does the network learn that a specific combination of features corresponds to a particular object class?	CNNs do not recognize objects from a single feature. They learn increasingly complex feature combinations across layers, and the final classifier learns which combinations are most indicative of each object class.

How was it decided to view the image in 16 different ways?	16 is just a design choice (hyperparameter). We choose the number of filters based on the complexity of the task and available computation.
At the end of output and scores how it's labelled form unknown image to Tiger/cat	At the end, the model produces a score for each class. Cat = 2.1 Dog = 0.5 Tiger = 4.8 Horse = 1.2 Cat = 5% Dog = 1% Tiger = 92% Horse = 2% The model chooses the class with the highest probability, so the image is labeled Tiger.
how do we decide that we have to apply reLu after convolution layer, like how we decide our next step	Apply ReLU after convolution because it helps the network learn complex, non-linear patterns. It is a design choice made when building the CNN.
Is this the right way I'm thinking: I have an image wanted to see it in various ways. There are 16 different ways and I max pull it to reduce the size and get rich representation of the picture by flattening it?	Yes. Multiple filters view the image in different ways, MaxPooling compresses the feature maps while keeping important information, and flattening converts them into a vector for classification.
Is CNN used just for images? or it can be used for other data types also?	While they are famous for 2D images, CNNs are used for any data where the order or spatial relationship of the data points matters. Example - Imagine a short, 1D horizontal line sliding left-to-right across a timeline. Sliding across an audio wave to detect the pattern of a specific spoken word
For word processing, CNN will need sequential processing right? And that is where we have RNN/LSTM?	Yess you are correct.
when we start training do those 16 filters already know how to detect edges or do thy learn it automatically?	The filters start with random values and automatically learn useful features (such as edges and shapes) during training.
So CNN is not that great for word processing ?	yes
But it also said it helps in some cases of NLP - which are those specific cases?	Imagine this Instead of sliding a square over pixels, a 1D CNN slides a horizontal "window" over a sentence, looking at 2, 3, or 4 words at a time (called "n-grams"). It scans the text looking for specific trigger phrases, regardless of where they appear in the paragraph.
How do we decide the weights of kernel in CNN and are they parameters or hyperparameters?	The weights (the numbers inside the filter) are Parameters. Hyperparameters are decided by the Human before training starts. (Example - I want 16 filters, I want 0.01 Learning Rate etc). Parameters are learned by the Machine during training. (Example - The actual decimal numbers inside those 3x3 filters, or the weights in the final MLP).
so when we design a CNN or any model, we decide how and what it will consider in the architecture be it RELu or any other function, or how many layers it will require etc. it is all part of architecture design, correct?	Yes, Correct, Just ensure youre using correct activation functions for correct layers, hidden/output layers.
So is it right understanding that CNN does learn filters with 16 layers in this example with each layer trying to learn different features like straight line, horizontal line and shapes etc.? If yes, what was the purpose of next layer to learn using 32 channels?	Yes, the first 16 filters learn simple features. The next layer with 32 filters combines those simple features to learn more complex patterns and object parts.

However even with RNNs multiple layers this is an issue and this is not specific to CNNs only. CNN are good with spatial data and RNN good with sequential data. So ... the first option in the quizz may also be a good answer ??? answer ...	Yes, in a broader sense the first option may also be reasonable because both CNNs and RNNs can struggle with long-range dependencies. However: If the quiz is specifically about CNN limitations, the expected answer is usually the one related to CNNs having a limited receptive field and needing many layers to capture distant information. So: The first option is generally true, but the quiz is likely testing the limitation that is most directly associated with CNNs.
selection of correct activation function depends on what factors? bias variance error?	Below is general selection criteria ReLU -> Most hidden layers Sigmoid -> Binary classification output Softmax -> Multi-class classification output Tanh -> Occasionally used when outputs need to be centered around zero
so convolution is like dividing the pics into different views??	You can think of convolution as scanning the image through a small window. The filter moves across the image, looking for specific patterns such as edges, textures, or shapes in each local region. It is not literally dividing the image, but it is processing it piece by piece.
Why this statement is partially right, that maxpooling make the CNN translation invariance.??	MaxPooling provides some translation invariance by making the model less sensitive to small shifts in feature location, but it does not make CNNs completely translation-invariant.
In CNN after several pooling followed by ReLu, how does it decided, it can be considered as flatten?	The decision to stop using Convolutions/Pooling and finally Flatten the data is based on Size and Meaning. if you flatten a massive image too early, the final Neural Network (MLP) will have hundreds of millions of weights, causing it to crash or overfit. Early in the network, 2D geometry matters. We keep adding Pooling layers until the physical dimensions of the feature map are very small, usually around 7x7 or 4x4 pixels. At this tiny size, flattening the data creates a manageable 1D array that the MLP can easily process without overloading your computer.
is number of channel directly propotional to features? In fact what does channels given in the example mean?	Yes When you first feed an image into a CNN, the channels refer to the physical colors. A standard color photograph is actually a stack of 3 separate grids of pixels: one Red, one Green, and one Blue. So, we say the input has 3 Channels. (A black-and-white image has 1 Channel). If a layer has 16 channels, it means the network is actively tracking 16 distinct features simultaneously.

Can a model be both underfitting and overfitting at different stages of training ?	Yes, For example - At Epoch 1, the network's filters are filled with random numbers. It has no idea what it is looking at. It performs terribly on the training data, and terribly on the validation data. Because it hasn't learned the basic patterns yet, it is Underfitting. As it trains, it starts finding the real patterns (edges, shapes, faces). The error on the training data drops, and the error on the validation data drops with it. The model is generalizing perfectly. If you keep forcing the model to train past the sweet spot, it runs out of broad patterns to learn. So, it starts memorizing the ultra-specific, useless noise in the training images. The training error keeps dropping to zero, but the validation error starts rising because the model is losing its ability to generalize. It is now Overfitting. So ensure youre using proper techniques like Dropouts and Early Stopping etc.
How do we know when to use CNN or not? Is there any Neural Network selection guidance?	CNN are generally used for Images, videos and spatial data
How do we decide the Kernel values ? And are they also updated ?	We do not decide the kernel values manually. The kernel values start with random initialization and are updated during training using backpropagation, just like the weights in a neural network.
if model learns patterns from kernels, wont they be lost if we flatten them at end?	No. Flattening does not destroy the learned patterns. Flattening simply converts those feature maps into a 1-dimensional vector so that the dense layers can use them for classification.
if model learns patterns from kernels, wont they be lost if we flatten them at end?	Wow this is such a a good question, Ansewr is No! Because by the time we flatten, the Convolutional kernels have already done the hard work of translating the 2D pattern into a numerical "score."
Between epochs, can trianing, validation or test data be interchanged. I mean some of the validation becomes part of trianing and some of trianing goes to validation.	No. Once your data is split before training begins, those splits must remain 100% frozen for every single epoch. Imagine you swap a few Validation images into the Training set on Epoch 5. The model's filters update and memorize those images. If you move those images back into the Validation set on Epoch 6, the model will score a 100% on them. But it didn't score 100% because it's smart. it scored 100% because it already saw the answer key! This is called Data Leakage. If your Validation set gets contaminated with Training data.
How are the kernel values updated then ? I mean is it also based on loss ?Can Professor answer this question	The kernel values are trainable parameters, just like weights in a fully connected layer. Initially they are randomly initialized. During training, the model computes a loss, and backpropagation calculates how much each kernel value contributed to that loss. Gradient descent (or an optimizer such as Adam) then updates the kernel values to reduce the loss. Over many iterations, the kernels gradually learn useful patterns such as edges, textures, and shapes.

So is it right understanding that CNN does learn filters with 16 layers in this example with each layer trying to learn different features like straight line, horizontal line and shapes etc.? If yes, what was the purpose of next layer to learn using 32 channels?	The first layer looks directly at raw pixels. Because a 3x3 or a 2x2 filter is so small, there are mathematically only a few useful patterns it can possibly find: a straight vertical line, a horizontal line, a diagonal edge, or a simple color gradient. The 32 channel layer would be more expansion, It looks at the edges found by Layer 1 and tries to combine them. Think about how many ways you can combine basic lines: Horizontal + Vertical = A sharp corner. Multiple diagonal lines = A triangular shape. Two curves joined together = A circle, or a curve maybe. Hope this is clear! :)
after flattening, when we feed it to network, is it one input against one neuron for say 14 rows?	No. It is a Fully Connected network, meaning every single input connects to every single neuron. Imagine your flattened data is a checklist of 14 numbers. You feed that into a Dense layer that has 10 Neurons. Because every neuron connects to every input, the math gets massive quickly. Just between those 14 inputs and 10 neurons, there are actually 140 separate connections (weights) being calculated!
Does spatial computing devices use CNN?	Yes, Spatial computing devices frequently use CNNs for computer vision tasks.
For example: A vision pro showing the images of people in genmoji form for FaceTime call.	Yes! In fact, spatial computing wouldn't exist without them. When you are wearing a headset and it shows a live, moving 3D avatar of you to the person on the other end of the call, it is running a massively complex pipeline of Neural Networks in real-time. Inside the Vision Pro, there are tiny infrared cameras pointing directly at your eyes and face. While you are talking, these cameras are capturing a constant stream of images. Every time you blink in a Vision Pro, a CNN is doing hundreds of rounds of Convolutions and Pooling in a fraction of a millisecond to detect that blink and recreate it on an avatar halfway across the world! This is exactly why companies like Apple build custom silicon chips (like the R1 chip) they need hardware specifically designed to do CNN matrix math as fast as physically possible.
How does overfitting is helped by drop outs can you verbally explain with an example once more ?	In a neural network, some neurons can become overly important. The model starts relying heavily on them and may memorize the training data. Dropout randomly switches off some neurons during training, forcing the network to learn alternative pathways and more general patterns.
on dropout, and during backpropogation is there a chance of not correcting the weights because one or more key feature got missing	Yes, a dropped neuron does not receive gradient updates in that step. But since dropout changes every iteration, important neurons are updated in other steps and the model learns more robust representations.
How do we choose the dropout neurons randomly? Can this remove properly good weights instead of bad weights?	Dropout randomly drops neurons regardless of whether they are good or bad. This forces the network to distribute learning across many neurons instead of depending on a few specific ones.
How do we choose which neuron needs to be drop	Instead, for each training step, the model randomly generates a mask and drops neurons according to the dropout rate. Example: Dropout = 0.2 Each neuron has a 20% chance of being dropped and an 80% chance of being kept. So: The neurons are selected randomly, not based on whether they are good, bad, important, or unimportant.

In a multi-layered neuron, is gradient calculated at each layer? If yes, how does the system knows the expected output at each layer of Neuron ?	Yes, the gradient is calculated at every single layer! Regarding Your second point The network does NOT know what the expected output of a hidden layer is. We only know the expected output at the very end. So, how do the middle layers learn if they don't have an answer key? They use Backpropagation and the Chain Rule of Calculus to pass the blame backward.
When we say, we switch off some neurons randomly, does it mean that we are turning off some filter functions? Can you give some correlation to the code?	Yes, in a CNN, Dropout effectively sets some neuron outputs (activations) to zero. It does not remove or delete the filters themselves. Code: Conv2D(32, (3,3)) ReLU() Dropout(0.5) With Dropout(0.5), roughly half of these activations are randomly set to zero during training
freezing a neurons is comes under fine tuning?	Yes. Freezing layers or neurons is a common fine-tuning technique where part of the model is kept fixed while the remaining parameters are trained.
why flatten layer is impt?	Flatten converts the 2D/3D feature maps into a 1D vector so that they can be fed into fully connected (dense) layers for classification.
then without train a model how we can use fine tune i am confused?	For example: ResNet was trained on ImageNet by researchers. You can download it and fine-tune it for medical images, cats vs dogs, etc. Short answer: Fine-tuning requires a pretrained model, but you don't need to do the original training yourself. You start from a model that has already been trained by someone else and continue training it on your data.
because as I know Fine-tuning IS training — just training on top of an already-trained model.	Fine-tuning is a form of training. The difference is that you start from a pretrained model instead of random weights. For example: Pretrained ResNet on ImageNet ↓ Fine-tune on Cat vs Dog dataset The model is still training (forward pass, loss, backpropagation, weight updates), but it starts from a much better initialization.
So feature maps dont run again and again between epocs..I understand the pipeline is as follows, Ones time feature map and multiple epocs training?	No. If you are training a model from scratch, the feature maps are recalculated every single time an image passes through the network, in every single epoch. If we are using Transfer Learning where we download a pre-trained CNN and freeze its filters so they never change then yes! We can run our entire dataset through the frozen CNN exactly one time, save the flattened feature maps to our hard drive, and then train our new MLP on those static numbers for 50 epochs. It saves computing power as well.
any limitation in Vocabulary or any number we can us	yes it depends on the number of words in your vocabulary
How can a model detect 3D spaces of the image if the flatten layer is converting them to 1D? Because the spacial computing devices produce 3D images.	The CNN learns spatial patterns before the flatten layer. Flattening happens only near the end, after the convolution layers have already extracted features and relationships in the image. Also, RGB images have 3 channels (Red, Green, Blue), which is different from true 3D spatial information.

How can a model detect 3D spaces of the image if the flatten layer is converting them to 1D? Because the spacial computing devices produce 3D images.	Good question Vamsi, and this is soo much advanced question as well, A spatial headset doesn't just feed the CNN a flat color image. It uses LiDAR or infrared projectors to measure exactly how far away every millimeter of your face is. The CNN looks at this "Depth Map" alongside the regular image. You are right that the Flatten layer destroys the physical grid of the image. But the neural network isn't trying to output a new grid! Instead, it is trained to output a 1D list of X, Y, and Z coordinates. Just an example that i can think is Nose_X: 2.5, Nose_Y: 1.1, Nose_Z: -0.8 The neural network's job ends at that 1D list. It hands those numbers over to a completely different piece of software, maybe a 3D Graphics Engine (Apple's RealityKit). The graphics engine reads that flat list of X, Y, Z coordinates, plots them in virtual space, and stretches a 3D digital skin over them to create the avatar! Hope this helps!
how are we repsenting the one hot for sentences	the words that are there will be represented by 1 in the index and 0 else
we only do 0 1 represenation for words ?	Yes. In one-hot encoding, each word is represented using only 0s and 1s.
The ohe hot representation shows the exact occurance of a word with a 1 why do we need wights then to evaluate wx+b	here x would be the one-hot rep, the feature set
What is bag of words representation? Is it different than one-hot vectors?	One-hot encodes a single word, while Bag of Words encodes an entire document by counting how many times each word appears.
how are we indexing the word in each sentence as 0 or 1 in one hot sentence?	So you can represent an array of V size, 1...V with the ith index being the ith word.
if 'happy' and 'joyful' have the same meaning, how do embeddings place them close together while one-hot encoding treats them as completely different?	One-hot encoding just gives every word a unique ID badge. If your vocabulary has 10,000 words, 'Happy' might be a 1 at position 5, and 'Joyful' is a 1 at position 8,000. To a computer, [1, 0, 0] and [0, 1, 0] are mathematically 100% orthogonal (different). The computer has no idea what the words mean; it just knows they are two different ID badges. But for embeddings - Because they share the same context, the network mathematically pushes their embedding coordinates closer together. They are no longer just binary ID badges but they are dense lists of decimal numbers (vectors) pointing to a specific coordinate in a massive 300-dimensional "Map of Meaning."
What is one hot vector representation?	A representation where if a word occurs in the sentence will be represented by 1 else 0
distance and angle both right?	generally we use cosine similarity but you may also use the Manhatten/Eucliedian dist
Is it distance b/w words?	cosine similarity between their one-hot representations
But why do we need any weights in one hot here when x only has the complete information ..no guessing work here..	One-hot encoding identifies the word, but the weights learn the meaning and relationships of that word for the task.
so there is a dictonary type of things for vectors, where words are vectors and as per the distance they are arranged in, so when model needs to make a relationship it picks most close distance words? like king -> man and queen -> woman	Yes, embeddings are like a learned dictionary of vectors where similar words are close together. The model uses these relationships to understand meaning and context, not just exact word matches.
Why to add hard coded synonyms?	prof is explaining, please wait patiently if the doubt is still there raise another doubt
Is to label the data as one called "Automobile" and list all the vehicle types?	yes sort of this is what signifies the option B

it means for option 1 -> cosine similarity 0 always for different words?	No. Different words do not automatically have cosine similarity 0. Similar words usually have high cosine similarity, while unrelated words tend to have values closer to 0.
why cosine similarity be zero, its zero when vectors are orthogonal	The model has not learned a strong relationship between those two words/concepts. They are neither particularly similar nor opposite.
what is pca	PCA used for dimensionality reduction. Please refer to Lecture 1
Lol.. I've a question, in the first case we hard coded in the second case why not? Eatery and Restaurant are fall under same labelled data right?	You can hard-code some synonyms, but it doesn't scale and misses context. Embeddings learn these relationships automatically from how words are used in real text.
The vectors dimensions of v	In One-hot vector dimensions is dimension of the vocabulary.
Dense embedding will now increase the size ? eventually cost ? and will the model will not become slow ?	Embeddings add trainable parameters, but they typically make the representation much smaller and more useful than one-hot encoding, so models are often more efficient overall.
What makes car and automobile to have numbers like .19 .21 values. I was reading that these values are decided based on set of n number of questions. E.g. guess the object based on questions. Is it correct.	The values are learned from word usage in large text corpora. Similar words like "car" and "automobile" end up with similar numbers because they appear in similar contexts, not because we manually define questions.
In dense representation, how do we maintain context i.e after searching for "fruit" then "apple" has a closer cosine similarity so is that with "phone" and "apple"	Static embeddings cannot fully distinguish "apple" (fruit) from "Apple" (company). Modern contextual embeddings use surrounding words (e.g., "fruit" or "phone") to generate different vectors for the same word in different contexts.
In dense vector every vector will have diff dimension?	no will have the same dimensions but will have reduced dimensions as compared to one-hot
do we need labels for vectors?	No. Embedding dimensions are usually unlabeled. The model learns them automatically from data.
why in the question then it was told that cosine similarity between two different words is 0 ..when weights are actually evaluating the relationship for a hot vector input	The statement "cosine similarity between two different words is 0" applies to one-hot vectors, not embeddings. After training, the embedding weights transform words into dense vectors whose cosine similarity can be high if the words are related.
how to define the dimension? is it a learned parameter?	The embedding dimension (e.g., 100, 300, 768) is chosen by the designer. The vector values within those dimensions are learned from the data.
In the current example, you still have vec(man); this if you add to vec(woman), won't it fail to deduce	You're thinking about the famous analogy: king - man + woman ≈ queen A common misconception is that the model literally "understands" man, woman, king, and queen. What's actually happening is: The embedding vectors have learned certain directions in space. For example, there may be a "gender direction": man → [...] woman → [...] and a "royalty direction": man → king woman → queen When we do: king - man + woman
distributed semantics give a value to each word and then we see the cosine similarity of those words?	Yes

So search engines use embeddings for predictive intelligence?	Yes, modern search engines use embeddings to understand meaning rather than just keywords. They compare the embeddings of queries and documents to retrieve semantically relevant results.
What is 300 dimension? May I know?	A 300-dimensional embedding is simply a vector of 300 numbers used to represent a word. The dimensions are chosen by the model designer, while the values are learned from data.
Is embedding model differs from LLM or both are same?	An embedding model produces vectors, while an LLM produces text. LLMs use embeddings internally, but they do much more than just create embeddings.
if a model is trained with finanical data and another model with medicine data. then the dimensions will be different for each model right?	Not necessarily The number of dimensions (e.g., an array of 300 numbers) is a setting chosen by the engineer before training even starts. You can absolutely train a Finance model with 300 dimensions, and a Medical model with 300 dimensions.
who decides where 'cat', 'kitten', and 'dog' are placed in the embedding space? Is it manually defined or learned automatically during training?	It is 100% learned automatically by the model during training. No human defines these positions. Before training, the model assigns every word in its vocabulary a completely random, junk coordinate in space. The model reads a content (maybe a paragrpah or a page of content sentence wise) The model looks at its current (wrong) guess. It then uses Backpropagation to tweak the numbers of the 'cat' embedding slightly so that next time, it guesses correctly. As the model does this billions of times, it notices that 'cat' and 'kitten' are always being swapped into the same blank spaces, surrounded by the same neighbors. (more examples - “meow”, “purr”, “fur”) The math forces the vectors for 'cat' and 'kitten' to migrate toward each other because that is the only way to minimize the error in its "Fill in the Blank" game.
embeddings start as random numbers. During training, how does the model decide that 'king' should become close to 'queen' and far from 'apple'?	During training, the model learns from word co-occurrence patterns. If "king" and "queen" frequently appear in similar contexts (e.g., royal, crown, palace), the model adjusts their embeddings to be closer together. If "apple" appears in very different contexts (e.g., fruit, tree, juice), its embedding gets pushed farther away. In short, words used in similar contexts end up with similar embeddings, even though they start as random numbers.
one question, LLMs get trained on text only or any kind of media? and embedding are part of the LLMs that contain values for each word and club those words with context near each other?	Traditional LLMs are trained on text, while newer multimodal models can learn from text, images, audio, and video. Embeddings are learned vector representations inside the model that capture contextual relationships between words, placing similar words closer together in the embedding space.
How do we know the number of dimensions?	its a hyperparameter
what are the practical use of skipgram and cbow	CBOW and Skip-gram are training strategies for learning word embeddings. CBOW is faster and works well for common words, while Skip-gram is slower but often learns better embeddings, especially for rare words.
In CBOW , using context word : output could be anything not specifically cat , so how is it faster ?	CBOW is faster because it uses multiple context words together to predict one target word. For example, given "The ___sat on the mat", the surrounding words provide strong clues, making the prediction easier and training more efficient. In contrast, Skip-gram uses one word to predict many context words, which requires more computations.

Can you help with real word examples for using the params Skip Gram vs CBOW?	CBOW is commonly used for large general-purpose corpora, while Skip-gram is preferred when learning good representations for rare or domain-specific words such as medical, financial, or scientific terms.
How do you decide, the number of dimensions in vector representation of any word?	The embedding dimension is a hyperparameter chosen by the designer. It is selected by balancing representation quality, computation cost, and performance on validation data.
so this Word2Vec trains model at runtime and capture vectors based on dimension no we provided?	yes vector_size
Is skip-gram or CBOW used by LLMs as well? Or do LLMs use more advanced methods of training the vast training dataset - given training is costly for LLMs?	no in case of transformers they use self-attention.
so Skip-gram or CBOW is a design choice? in what situations do we choose Skip-gram vs CBOW? is this based on precision and recall, when we need more precision we will choose Skip-gram?	Yes. It is a design choice that depends entirely on the nature of your training data. If your goal is to predict the center word given the surrounding context words. and you have a massive dataset and need fast training. then CBOW If your goal is to predict the surrounding context words given a single center word. and you have a smaller dataset or need to capture rare words. then skip-gram. This is how i remember - you want the most precise, high-quality embeddings for rare/specialized terms, you choose Skip-gram. you want a fast, reliable model for a massive general-purpose corpus (like the whole internet), you choose CBOW.
How does word2Vec know that king and man are closer to one another? does this model alreadyhas context?	No, Word2Vec does not start with context knowledge. It starts with random embeddings and learns from the training text. If "king" and "man" often appear in similar contexts, Word2Vec gradually adjusts their embeddings to be closer together. The context is learned from the data, not built in beforehand.
Why option D is wrong here ?	Prof is adressing this, kindly listen once, if still you have doubt you can raise a question
how does word2vec is adjusting the it weights with king and man.. if multiple sentences in the training we mention about king and man then that means word2vec adjusting the weights? what if we have few sentences in training data says about king and man together in sentence?	Yes, Word2Vec updates its weights every time it processes a training example. The more often words appear in similar contexts, the stronger the adjustment. Even if two words rarely occur together, they can still become similar if they frequently appear in similar surrounding contexts.
Can Word2Vec predict semantically as its not designed for it? its predicting words given multiple sentences or words. If above understanding is correct- there would be a high incorrect prediction. Please elobrate on this.	Yes. You are correct that Word2Vec is designed as a predictive machine (e.g., "predict the next word"), but the "semantic" ability to understand meaning is the byproduct of that prediction. The model is not trying to learn the definition of "cat." It is just trying to minimize the error on its "Fill in the Blank" game. Also, You are right! If you ask Word2Vec to write a sentence or predict the exact next word, it is often incorrect and nonsensical. It is a terrible writer. It does not know grammar, syntax, or long-term logic. BUT BUT BUT, remember this It is an amazing dictionary. Even if it can't write a paragraph, it has successfully mapped the relationship between words into a numerical space.
why in the question then it was told that cosine similarity between two different words is 0 ..when weights are actually evaluating the relationship for a hot vector input	The statement "cosine similarity between two different words is 0" refers to the raw one-hot representations. The learned weights (embedding matrix) have not yet been used. Once the weights are applied, the resulting embeddings can have non-zero and often high cosine similarity.

Are there any models which can understand sarcastic sentences?	sarcasm remains challenging because it often relies on tone, intent, and background knowledge that may not be explicitly stated in the text. However, Modern Transformer-based models can often recognize sarcasm by analyzing context rather than individual words.
if I understood this correctly or maybe wrong, had the model received data from agriculture weekly along with Bloomberg it would have not collapsed ALL Bank words to one vector?	No. Even with Bloomberg + agriculture data, Word2Vec would still give "bank" a single vector. It would learn a blended representation of both meanings rather than separate meanings. Contextual models like Transformers solve this by generating different vectors for different contexts.
bank for river context and bank for finance context will have same static meaning. so we are using transformers to make it dynamic?	Yes. Static embeddings give "bank" one fixed meaning. Transformers generate context-dependent embeddings, allowing "bank" to mean finance in one sentence and river edge in another.
Even TF-IDF technique is sparse?.	Yes. TF-IDF is a sparse representation because most dimensions are zero for any given document.
Forn transformer, we add positional encoding and contextual encoding to original bank vector. So does bank vector originally fetched contain all meanings from financial and river perspective?	The initial embedding for bank is a general-purpose representation learned from all its usages.
Attention mechanism helps to get this dynamic vectors?	Yes Attention mechanism updates the initial representation of vectors based on context.

could you please expalain CBOW by comparing with word2vec library?	Inside Word2Vec, you choose one training style:
	from gensim.models import Word2Vec # CBOW
	model = Word2Vec(sentences, vector_size=100, window=5, sg=0)
	# Skip-gram
	model = Word2Vec(sentences, vector_size=100, window=5, sg=1) Here:
	sg=0 means CBOW sg=1 means Skip-gram
	CBOW idea:
	I love ___learning Context words: I, love, learning
	Target word: machine
	CBOW learns by using surrounding words to predict the missing middle word.
	So:

What if a model requires both financial and medical data and the same word can have multiple meanings in the same data ?	while creating vector representations, the model often enriches context from surrounding words. This will help the model differentiate between financial and medical context and assign different representations for the same word.
If one hot vector has that much limitation then is there any practical use of it in real world?	One-hot encoding is still widely used for small categorical variables and class labels. It becomes problematic mainly when the vocabulary is very large or when semantic similarity between words matters.
For transformer, we add positional encoding and contextual encoding to original bank vector. So does bank vector originally fetched contain all meanings from financial and river perspective?	Yes. The original embedding for "bank" is a single learned vector containing information from all its usages. The Transformer then uses context to produce a more specific meaning for that particular sentence.
so in Skip-gram forced to learn its meaning	Yes. Skip-gram forces a word's vector to become informative enough to predict its surrounding words, which is how it learns meaning from context.
is word embedding only training time activity where are vectors are frozen, or can they change during inference dependeing on data.	Static embeddings (Word2Vec) are frozen after training. In Transformers, the weights are frozen during inference, but the final word representation changes dynamically based on the surrounding context.

How is the distance between two words actually calculated? Like say Bird and Tree or even BIRD and TrEE? Is there some sort of algorithm? I am aware that we have libraries which do this thankfully and we need not worry about it. However, I query out of curiosity ...	The distance is usually calculated using cosine similarity between the word vectors. Similar words have vectors pointing in similar directions, giving a higher cosine similarity. Like Bird and Tree or even BIRD and TrEE have similar consine similarity. In training phase we lower case all the words in the document to maintain consistency.
What information is lost during max pooling phase?	Max Pooling keeps the strongest feature response while discarding weaker activations, trading detail for efficiency and robustness.
In case of transformer, original vector of word we get using Word2Vec and then contextual encoding is added on top of it?	No. Transformers usually learn their own embeddings. The contextual representation is created by the self-attention layers, not by taking a Word2Vec vector and adding context on top.
what is the purpose of multiple nodes in a single hidden layer? and all the weights are pre initialized before traning begins?	No. Transformers usually learn their own embeddings and then use self-attention to create context-dependent representations. They do not normally start with Word2Vec embeddings.
Will embeddings remain important in future AI systems, or could they be replaced by another representation?	Embeddings are likely to remain a core idea, but future AI systems may use more advanced, dynamic, or multimodal representations instead of today's simple word embeddings.
in any specific real life exercises if there is a chance for both frequent & rare topic may occur, in those cases, how to make the balance between 2 approaches while using word2vec	When both frequent and rare words matter, Skip-gram is often the safer choice because it learns better representations for rare words while still working well for common words.
If a CNN can detect edges anywhere in an image, does that mean it automatically becomes rotation invariant too?	No. CNNs are somewhat translation-invariant, but they are not automatically rotation-invariant because rotating an object changes the patterns seen by the filters.
But why would the words result in vectors pointing in any direction at all in first place ? Even if they do point in a direction, what logic are they using to point towards a direction? Would they keep pointing in same or similar direction for the same two words?	Vectors start random. Training repeatedly adjusts them so that words used in similar contexts move into similar directions. The exact coordinates are not important; the relative positions and directions between words are what matter.
How do we decide on the datatype of the pytorch that is converted from numpy value ?	By default, PyTorch uses the same dtype as the NumPy array. You can override it by specifying dtype= when creating the tensor.
CUDA available is coming as FALSE, what should I do?	Please connect the CUDA kernel in Google Colab
what are the other dtype:dtype : torch.float32?	torch.float16 # half precision torch.float32 # most common for deep learning torch.float64 # double precision torch.bfloat16 # used in modern AI training
Are those 4 values , x-train , x-test , y-train , y-test values ?	Yes. The four values are training features (X_train), test features (X_test), training labels (y_train), and test labels (y_test).
Can we name the dimensions in a Vector or Tensor ? like how we define row and columns in pandas	Yes. Tensors can have named dimensions (e.g., batch, channel, height, width), but standard PyTorch tensors usually only store dimension sizes, not labels like Pandas.
Then what is main difference betw arange and linspace?	arange() - You specify the step size. linspace() - You specify the number of points.
what is the meaning of . after number in the output of the tensor?	1. means 1.0. The dot indicates the tensor is storing floating-point values rather than integers.
Is it possible to store integers? If yes how?	Example: import torch x = torch.tensor([1, 2, 3], dtype=torch.int32) or x = torch.tensor([1, 2, 3], dtype=torch.int64) You can check the type: print(x.dtype) Output: torch.int32 or torch.int64 Short answer: Yes. Use an integer dtype such as torch.int32 or torch.int64 when creating the tensor.

where will we apply all this in M/L?	In ML, floats are used for data and model parameters, integers are used for labels and indices, and booleans are used for masks and conditions.
Can you please explain agaun the 4D tensor what image looks like example ?	A 4D tensor for images is (batch_size, channels, height, width). For example, (32, 3, 224, 224) means 32 RGB images, each of size 224×224 pixels.
what is shape?	Shape describes the size of a tensor along each dimension. For example, a tensor with shape (2, 3) has 2 rows and 3 columns
Can you please explain agaun the 4D tensor what image looks like example ?	A 4D tensor is simply a stack of images. Each image contains 3 color matrices (R, G, B), and the batch dimension tells you how many images are processed together.
is pytorch impt for genAI model and agentic ai learning?	Yess
are we doing multipe ephocs with the same data ?	Yess
what is Adam?	Optimizer
if we apply the Regularization, then gradients will be vanized ?	No. Regularization adds to the gradient and encourages smaller weights. It does not directly cause vanishing gradients.
error is minimized after passing one data point in one epoch or after passing through all data points in one epoch or after all epochs?	In most deep learning training, weights are updated after every batch of data. The loss is gradually minimized across batches, epochs, and the entire training process.
what is Tensor RT optimization?	TensorRT is an NVIDIA library that makes trained deep learning models run faster and more efficiently during inference.